

A study of JAUS compliant mobile robot control by using Tangent Bug Algorithm

Daisuke Minegishi, Kzuyuki Kobayashi and Kajiro Watanabe
Faculty of Engineering, Hosei University
3-7-2 Kajinocho, Koganei, Tokyo 184-8584, Japan
minegisi21@gmail.com

Abstract— Obstacle avoidance while keeping lane guide is a key technology for intelligent mobile robots. To navigate mobile robot under unknown environment without collision to obstacles, Tangent bug is one of useful and practical path planning algorithm for mobile robot. Since Tangent bug requires only range sensor with less computational power comparing other path planning algorithm. Simulation examples of proposed path planning algorithm are shown to demonstrate validity of a proposed method. And also, in order to apply our path planning algorithm to actual mobile robot, we define JAUS compliant path planning interface for simulation. Employing JAUS compliant interface enables interoperability between other types of mobile robot.

Index Terms— Tangent Bug, IGVC, JAUS

I. INTRODUCTION

The ARL laboratory at Hosei University has been developed intelligent mobile robot for participating autonomous challenge at the Intelligent Ground Vehicle Competition (IGVC) conducted by the Association for Unmanned Vehicle Systems (AUVS).

In the competition, a mobile robot must navigate lane guided course around obstacles under the outdoor environment avoiding the obstacles while keeping on the track course using vision and range sensing devices. In order to track course while avoiding collision to obstacles, appropriate path planning is necessary. There are various types of path planning method is proposed and used for mobile robot in IGVC autonomous challenge.

Most popular approach in IGVC autonomous challenge is based on grid map approach. For example, Team Wunderbot is applying O^3 method which is done in two steps: explicit using GPS coordinate (before motion) and implicit using Voronoi polygons (during motion) [1]. Team MikeRobot is applying A-star method which computes the lowest risk path from a designated starting point and a designated ending point in a weighted grid [2].

These grid map approach tend to consume huge memory for grid map generation.

The Tangent Bug algorithm as originally developed by Kamon, Rivlin, and Rimon [3] is designed for a point robot, with sensing capabilities, in a two-dimensional environment. This algorithm

requires relatively less memory comparing to above grid map based approach.

However the problem of original Tangent Bug algorithm is derived based on a point robot, it is not take into account the size of actual mobile robot. Depending on situation of obstacles, Tangent Bug algorithm may generate impractical path to avoid obstacles. To overcome the problem of Tangent Bug algorithm, we newly develop and propose a new robot size-aware Tangent bug algorithm which includes size of actual mobile robot.



Fig 1(a) Obstacle zone of IGVC rally

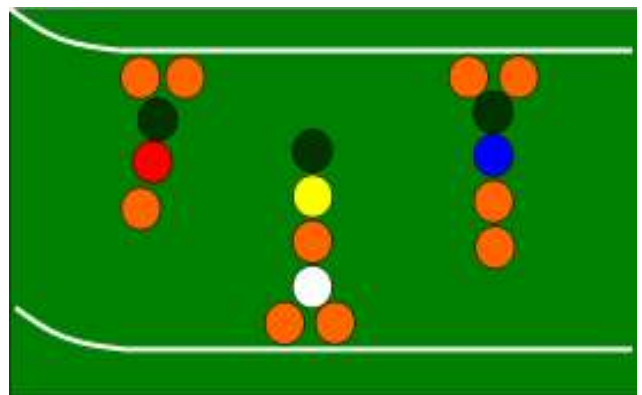


Fig 1(b) Outline of obstacle zone

Fig. 1: Obstacle zone

II. ASSUMPTIONS AND PROBLEM DESCRIPTION FOR PROPOSED MODIFIED TANGENT BUG ALGORITHM



Fig. 2: Between obstacles

Fig. 2 shows typical example of complex obstacle course in IGVC autonomous challenge. There are various colored barrels are set on the course. To find barrels accurately, we use laser range scanner to view horizontal arrangement of obstacles. Scanned range information from laser range scanner is mobile robot centered polar coordinate system.

In order to apply proposed modified Tangent bug algorithm to our mobile robot, we define following variables that illustrated in Fig.3.

[Variables]

(x, y) : x-y coordinates (for mobile robot and edge of laser position)

$P_r(x, y)$: Position of laser range center

$P_i(x, y)$: Position of boundary between laser hitting obstacle and laser not doing one. ($i = 1, 2, \dots, n$)

$P_d(x, y)$: Position of destination

d_1 : Distance from $P_r(x, y)$ to $P_i(x, y)$

d_2 : Distance from P_i to $P_d(x, y)$

d_3 : Distance from $P_{r(x,y)}$ to $P_d(x, y)$

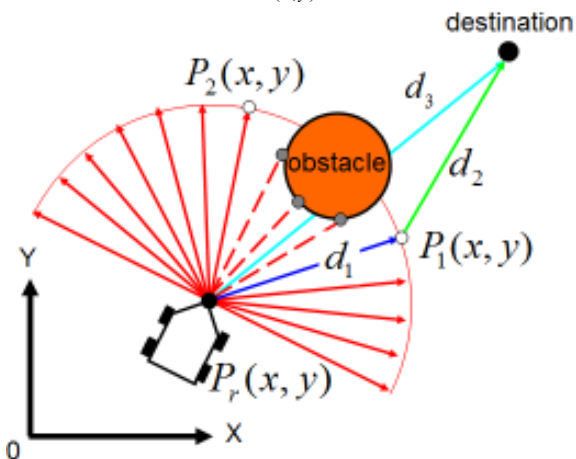


Fig. 3 Moving direction in Tangent bug algorithm

Assumptions:

A1) Dynamically moving object such as walking human and moving other vehicles are not take into account.

A2) Two wheeled mobile robot which equipped on-board range sensor which can measure front horizontal profiles around maximum 180 degrees.

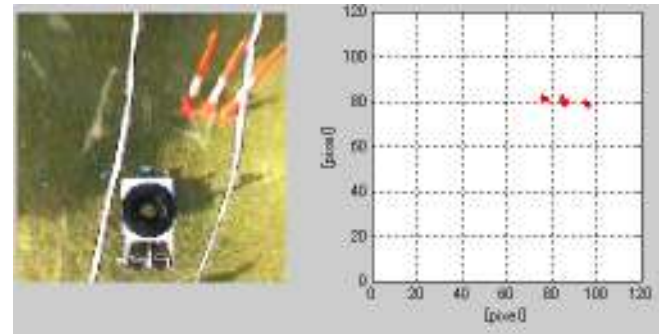
Assumption (A1),(A2), we only consider static environment such as IGVC autonomous challenge course.

The problems considered here are:

P1) How to recognize obstacles gap from range sensor information.

P2) How to generate appropriate path regardless of obstacles arrangement by using proposed robot size aware Tangent Bug algorithm.

We cluster lasers because two or more lasers hit one obstacle. Figure 5 shows that mobile robot recognized obstacles by laser range scanner. Figure 4(a) shows captured image. Figure 4(b) shows that two or more lasers hit one obstacle. Figure 5 shows the outline chart.



(a) Image capture

(b) laser hitting obstacles

Fig. 4 Recognized obstacles by laser range scanners

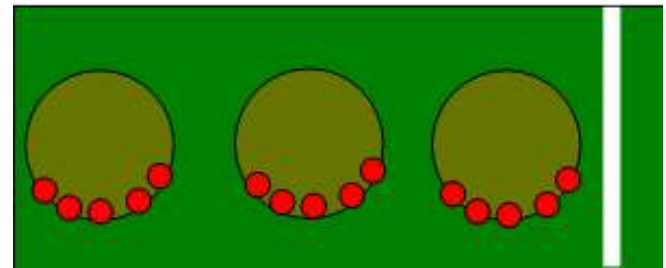


Fig. 5 Diagrammatic illustration of irradiation

III. PROPOSED MODIFIED TANGENT BUG ALGORITHM

Depending on obstacles arrangement on the course, the mobile robot may not path through between obstacles due to mobile robot size itself.

The major contribution of proposed modified tangent bug algorithm can handle above problem of original tangent bug algorithm which assumes a point robot.

The flow chart of the proposed modified tangent bug algorithm can be described as in Fig 6. The procedure is as follows and, Fig 3 shows flow chart of new tangent bug algorithm.

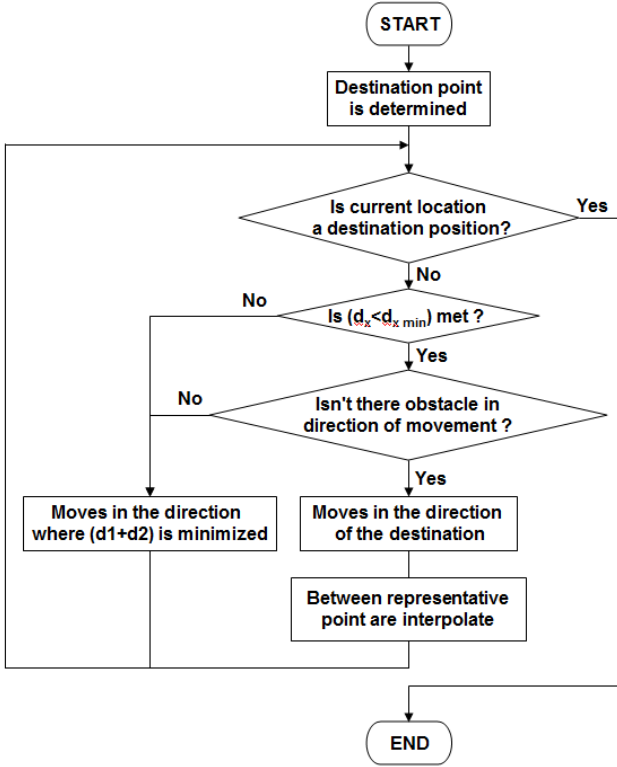


Fig. 6 Flow chart of new Tangent Bug Algorithm

Step 1) Destination point is determined.

Step 2) Direction of movement is determined as $P_{i(x,y)}$ that

$(d_1 + d_2)$ is minimized.

Step 3) While mobile robot recognizes obstacles, it skirt around the obstacle boundary until the leaving condition is met.

The leaving condition is as follows:

- We defined function $P_{x min}$ which is P_x when d_3 is minimized. And, when $d_x < d_{x min}$ is met.
- When are there no obstacles in laser range of the direction of movement.

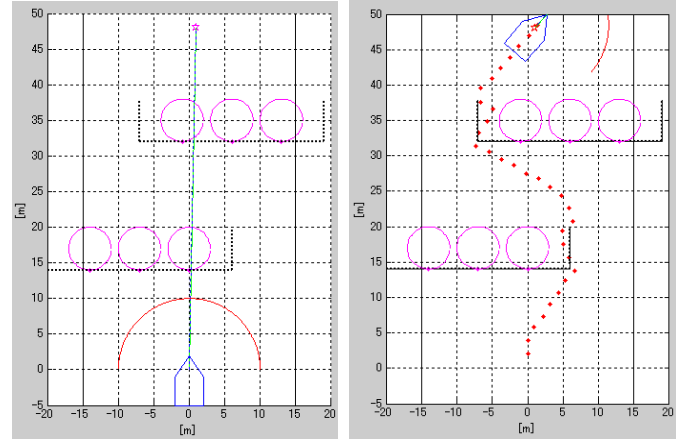
Step 4) We define representative point of obstacle as center of laser hitting obstacles.

Step 5) We interpolate between representative point by several kinds of methods.

IV. SIMULATION RESULT

To verify the proposed new Tangent bug algorithm, simulations are examined by using actual data that is acquired in IGVC autonomous challenge. The assumed size of mobile robot is 0.5(W)x1.2(D) meter. The obstacles are aligned as shown in Fig.7 and obstacle's diameter is about 6 meter. Minimum distances between obstacles are about 1 meter.

Fig 7 shows simulation of new Tangent bug algorithm. Fig 7 (a) shows mobile robot position initial position and target destination that is shown in star mark. Fig 7 (b) shows trajectory of mobile robot.



(a) Before simulation (b) Trajectory of mobile robot
Fig. 7 Simulation of new Tangent Bug Algorithm

In order to compare performance of proposed interpolated clustering, we apply straight line based interpolation and c-spline based interpolation is applied for simulations.

Fig 8 shows typical example of difference between straight line based interpolation and c-spline based interpolation. Fig.8(a) shows original alignment of obstacles and Fig. 8(b) shows straight line based interpolation and Fig.8(c) shows C-spline based interpolation results.

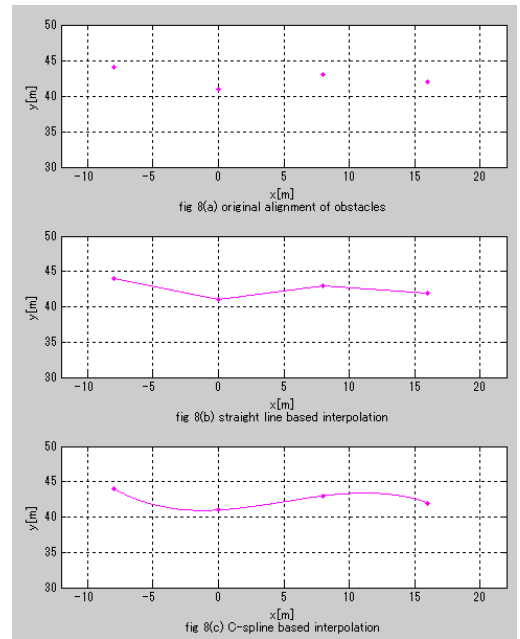


Fig. 8 Interpolation of between obstacles

In order to compare effect to different types of interpolation method, we simulate obstacle avoidance by applying different moving direction. Fig 9 shows simulation result of obstacle avoidance. Fig 9(a) shows simulation result that detected obstacles are applying as straight line interpolation. Fig.9(b) shows simulation result that applying c-spline based interpolation. Fig.9(c) shows different direction approach that applying stright line interpolation. and Fig.9(d) different direction approach that applying c-spline based interpolation. Fig. 9(e) shows each trajectory comparison between straight line interpolation and c-spline based interpolation.

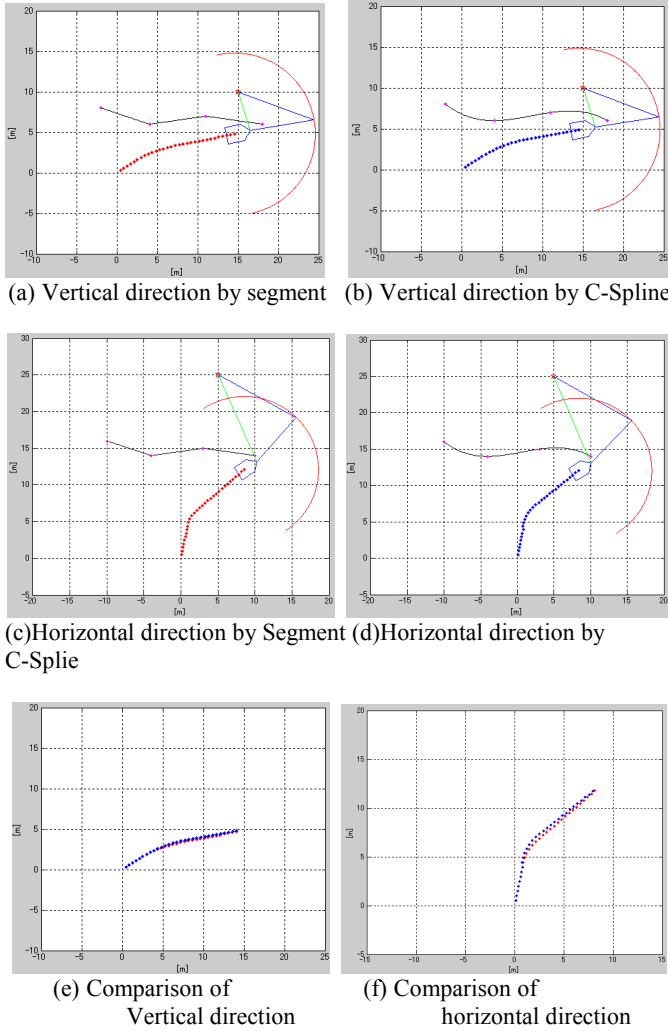


Fig. 9 Simulation result of interpolating obstacle

V. IMPLEMENTATION OF OUR JAUS COMPLIANT PATH PLANNING DRIVER

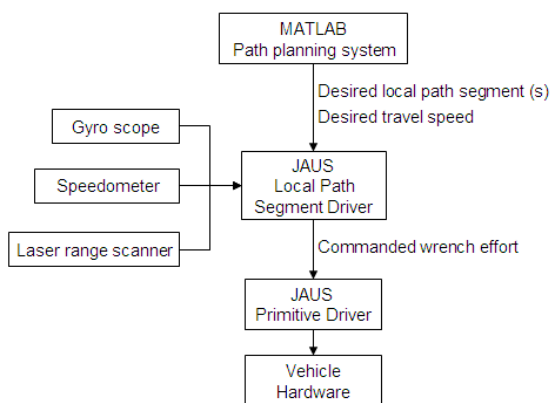


Fig. 10 Local Path Segment Driving Block Diagram

Table 1 Append SetLocalPose presence vector

Input Messages	Expected Response	Required Fields (PV)
Set Local Pose	<none>	X, Y, Yaw, laser_x & laser_y [67 Decimal, 0643h]

Fig.10 illustrates structure of our JAUS compliant path planning driver. SetLocalPose in JAUS original command, is used as to set destination position for our algorithm. In order to implement our algorithm, we append laser range profile (laser_x and laser_y) in SetLocalPose presence vector that is shown in Table. 1.

VI. CONCLUSION

In this paper, we proposed a new Tangent bug algorithm for IGVC autonomous challenge. In order to take into account size of actual mobile robot, we introduce new interpolation based clustering method according to obstacles location.

Straight line based interpolation and C-spline based interpolation are compared for proposed clustering method.

The simulation results conclude no distinct difference except for processing time. The validity of the proposed new Tangent bug algorithm was examined by simulation.

We find straight line based obstacle interpolation achieve enough performance for obstacle avoidance for proposed tangent bug algorithm.

VII. ACKNOWLEDGMENT

A part of this research was supported by Grants-in-Aid for Scientific Research (C), No. 22300201 from Japan Society for the Promotion of Science.

REFERENCES

- [1] LOUISIANA STATE UNIVERSITY MIKEROBOT, "INTELLIGENT GROUND VEHICLE COMPETITION TEAM 2008", Intelligent Ground Vehicle Competition Design Report, 2008
- [2] Elizabethtown College, "Wunderbot IV", Intelligent Ground Vehicle Competition Design Report, 2008
- [3] Kamon, I., Rivlin, E. and Rimon, E., "A New Range-Sensor Based Globally Convergent Navigation Algorithm for Mobile Robots", in Proc. IEEE Conf. Robotics Automat, 1996.